

PRODUCT PARK

DATAKOM

Dossier technique — Réécriture et sécurisation d'une application monolithique

Analyse de l'existant, architecture cible, choix techniques et sécurité

Séminaire — Réécriture & sécurisation d'une application monolithique

Stack cible : Vue 3 (Vite) · Spring Boot · PostgreSQL · JWT

Sommaire

(Clic droit sur le sommaire → « Mettre à jour les champs » pour afficher la pagination à l'ouverture du document.)

1. Contexte et objectifs

DataCom est le module de Product Park dédié à l'enregistrement puis à la validation de produits avant leur mise en circulation sur le marché. L'application existante est un monolithe Java/JSP développé sans couche de sécurité applicative digne de ce nom : pas de contrôle de rôle, mots de passe en clair, injections SQL multiples, et une confusion générale entre affichage, logique métier et accès aux données.

Ce dossier documente la démarche complète de reprise du projet : l'analyse de l'existant et de sa dette technique, la stratégie de refonte retenue, l'architecture cible (frontend Vue 3 / backend Spring Boot / PostgreSQL / JWT), le modèle fonctionnel du workflow produit, et enfin le détail des briques de code les plus importantes de la nouvelle implémentation.

Objectifs poursuivis :

- Séparer strictement les responsabilités (couches Controller / Service / Repository, DTOs, mapper).
- Éliminer les failles de sécurité identifiées (injection SQL, mots de passe en clair, absence de contrôle de rôle, XSS, fuite d'informations sensibles).
- Conserver les comportements fonctionnels validés (workflow en 4 steps sans retour arrière) tout en corrigeant les bugs (rôle USER superflu, step non fiable côté serveur).
- Documenter et justifier chaque choix technique pour qu'il puisse être repris et expliqué en maintenance.

2. Analyse de l'existant

2.1 Présentation du monolithe

L'application d'origine est un monolithe Java exécuté dans un conteneur Tomcat, sans découpage en couches : les servlets (LoginServlet, ProductServlet) mélangent routage HTTP, logique métier et accès direct à la base de données via JDBC. L'affichage est assuré par des pages JSP (product.jsp) qui réinjectent directement les données reçues, sans échappement.

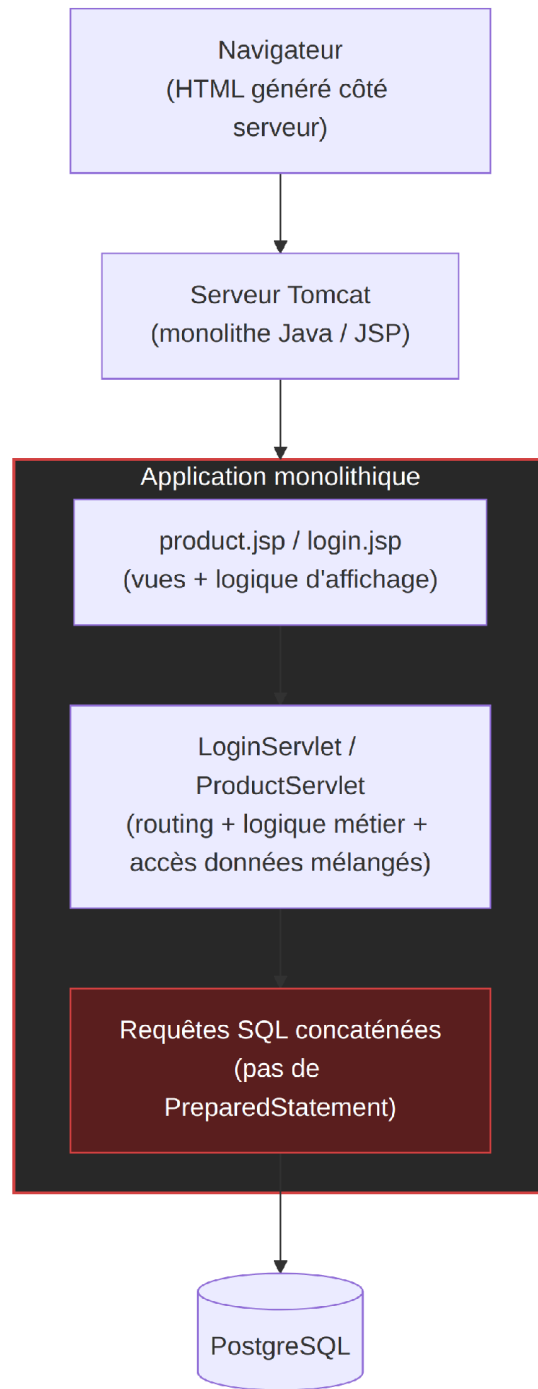


Figure 1 — Architecture de l'application existante : un seul bloc concentrant vues, contrôleurs et accès SQL bruts.

2.2 Dette technique et failles de sécurité identifiées

L'audit du code existant fait remonter 16 problèmes, du bug fonctionnel à la faille de sécurité critique. Ils sont classés par gravité ci-dessous.

Failles de sécurité critiques

#	Problème	Constat dans le code
1	Injection SQL généralisée	LoginServlet et ProductServlet concatènent toutes les requêtes SQL avec des paramètres bruts ; aucun PreparedStatement dans tout le projet. Bypass d'authentification trivial (ex. login: admin' --).

#	Problème	Constat dans le code
2	Aucun contrôle de rôle serveur	doGet/doPost ne vérifient que user != null (session active), jamais user.getRole(). Un rôle USER (censé n'être qu'un lecteur) peut créer et valider des produits.
3	Mot de passe en clair en base	INSERT INTO users(...) VALUES ('admin','admin',...) — aucun hash, aucun sel.
4	Mot de passe loggé en clair	LoginServlet fait System.out.println(sql) avec le mot de passe visible dans la requête SQL affichée sur stdout.
5	XSS stocké	product.jsp réaffiche name, description, category... avec <%=...%> sans échappement HTML, cumulé à l'injection SQL sur les mêmes champs.
6	Fuite d'informations côté client	Un bloc « Debug information » affiche session.getAttribute("user") (donc potentiellement le mot de passe) directement dans le HTML rendu.
7	Absence de protection CSRF	Les actions save/validate/new sont de simples formulaires POST/GET sans token, donc exploitables par CSRF.
8	Credentials DB en dur	Database.java code en dur USER="postgres", PASSWORD="postgres" dans le code source versionné.

Bugs fonctionnels et qualité de code

#	Problème	Impact
9	step non fiable côté serveur	Integer.parseInt(step) fait confiance à un champ <input type="hidden"> modifiable côté client : rien n'empêche de forcer un step arbitraire par URL.
10	Stack trace exposée au client	Tous les catch font e.printStackTrace() puis relancent une exception non gérée, potentiellement affichée telle quelle par Tomcat.
11	Bouton « Save » redondant	« Save » et « Save & Next » appellent tous deux product?action=save : chaque changement de step est déjà sauvegardé automatiquement.
12	3 rôles au lieu de 2	Le rôle USER, censé n'avoir aucun droit de création/validation, se retrouve avec les mêmes permissions que ADMIN et VALIDATOR faute de contrôle.
13	Classe orpheline dupliquée	User.java et UserCopy.java coexistent ; le code utilise UserCopy partout, User semble mort.
14	Code mort commenté	Un ancien doPost entier est laissé en commentaire dans LoginServlet.java (« NE PAS SUPPRIMER ») au lieu d'être dans l'historique Git.
15	ResultSet mal gardé	rs.getString("validation") est appelé hors du bloc if(result.next()) : exception non gérée si l'id n'existe pas.
16	Fuite de connexions DB	Aucune fermeture de ressources dans les blocs catch, pas de finally ni try-with-resources.

Conclusion de l'audit : l'existant ne peut pas être corrigé « à la marge » — l'absence totale de couche de sécurité et le mélange des responsabilités imposent une réécriture complète, mais en conservant scrupuleusement les comportements fonctionnels validés (workflow en 4 steps, sans retour arrière).

3. Stratégie de refonte et choix techniques

La stack cible retenue sépare complètement le frontend et le backend, communiquant via une API REST authentifiée par JWT.

Composant	Choix retenu	Justification
Frontend	Vue 3 (SPA) + Vite	Écosystème léger, build rapide, bien adapté à une SPA consommant une API REST ; Vite remplace webpack pour un dev server quasi instantané.
Backend	Java + Spring Boot (API REST)	On conserve Java (montée en compétence de l'équipe) mais on adopte un framework qui impose une architecture en couches et fournit Spring Security nativement.
Base de données	PostgreSQL (conservée)	Aucune raison de migrer : le problème n'était pas la base mais la façon d'y accéder (SQL concaténé). Spring Data JPA règle ce point.
Authentification	JWT (stateless)	Compatible avec une API séparée du front ; pas de session serveur à gérer ; le rôle est signé dans le token et revérifié à chaque requête.
Conteneurisation	Docker (conservé)	Simplifie le déploiement des deux applications séparées ; à affiner (docker-compose) une fois l'architecture stabilisée.

3.2 Pourquoi Spring Boot règle les problèmes de l'existant

- Spring Security : gestion des rôles (ADMIN/VALIDATOR) via annotations (@PreAuthorize, hasRole(...)), appliquée systématiquement côté serveur — élimine le bug n°2 (aucun contrôle de rôle).
- Spring Data JPA : requêtes via repository + entités, donc plus de concaténation SQL manuelle — élimine le bug n°1 (injection SQL).
- BCrypt (fourni par Spring Security) : hash des mots de passe nativement — élimine le bug n°3 (mot de passe en clair).
- Architecture en couches imposée par le framework : Controller / Service / Repository — élimine le mélange des responsabilités de l'existant.
- API REST : Spring Boot expose naturellement une API JSON, cohérente avec un frontend Vue séparé.

3.3 Point de vigilance assumé : le JWT et sa révocation

Un JWT classique reste valide jusqu'à expiration, même après une « déconnexion » côté client (le logout ne fait que supprimer le token du localStorage, cf. endpoint POST /api/auth/logout). Ce risque est assumé pour la version actuelle, avec une compensation directe : une durée de vie de token courte, et la case échéante l'introduction ultérieure d'une stratégie de refresh token ou d'une blacklist serveur si le besoin se confirme.

Le corollaire de ce choix (stockage du token en localStorage plutôt qu'en cookie) est que la protection contre le vol de token repose entièrement sur l'absence de XSS côté frontend : c'est pourquoi l'échappement

systématique du contenu utilisateur (bug n°5 de l'existant) devient la ligne de défense principale de la nouvelle architecture, et pas un simple détail de présentation.

4. Architecture cible

4.1 Vue d'ensemble

Les deux applications sont désormais des dépôts séparés (product-park-datacom-front-end et product-park-datacom-back-end), qui communiquent exclusivement en HTTP/JSON, avec le JWT porté dans l'en-tête Authorization: Bearer <token>.

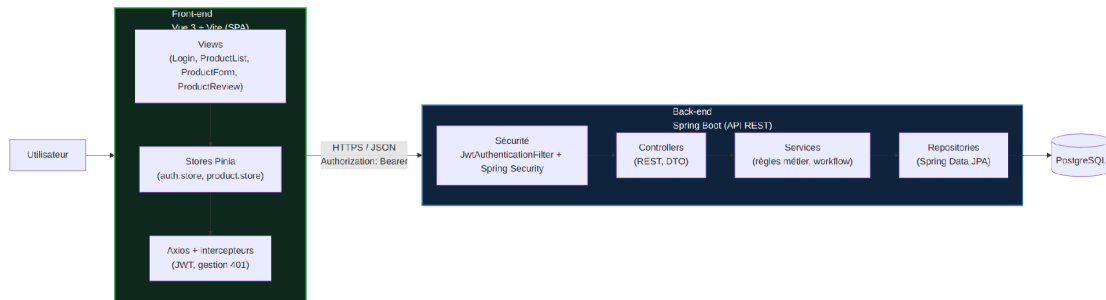


Figure 2 — Vue d'ensemble de l'architecture cible : SPA Vue 3 côté client, API REST Spring Boot côté serveur, PostgreSQL en persistance.

4.2 Backend : architecture en couches

Le découpage en packages (controller / service / repository / model / dto / mapper / security / exception) rend chaque responsabilité isolée et testable indépendamment.

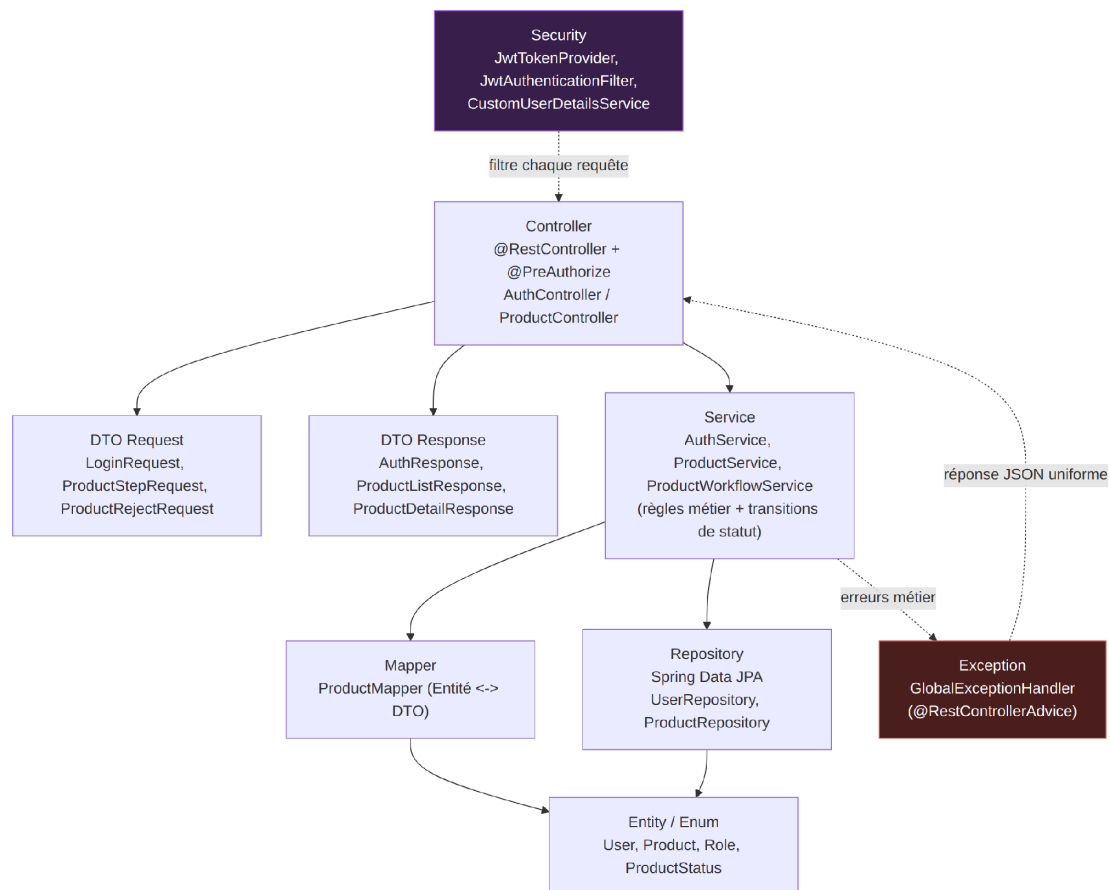


Figure 3 — Circulation d'une requête à travers les couches du backend : sécurité JWT en amont, DTOs en frontière d'API, gestion d'erreurs transverse.

Ce découpage règle directement plusieurs bugs de l'existant :

- Les DTOs empêchent d'exposer les champs sensibles (mot de passe) et empêchent le client de forcer des champs qu'il ne devrait pas contrôler (status, id, createdBy...).
- ProductWorkflowService centralise la logique des steps : plus possible de « sauter » un step ou de forcer un step invalide depuis le front, car c'est vérifié côté serveur avant chaque transition (règle le bug n°9).
- Role en enum + Spring Security (@PreAuthorize) rendent le contrôle de rôle systématique, plus un oubli comme dans l'existant (règle le bug n°2).
- GlobalExceptionHandler (@RestControllerAdvice) garantit qu'aucune stack trace ne fuite jamais dans une réponse HTTP (règle le bug n°10).

4.3 Frontend : structure de la SPA

Le frontend suit une organisation classique Vue 3 + Pinia : des vues par écran, des composants réutilisables par step de formulaire, des stores Pinia pour l'état partagé (authentification, produit en cours d'édition), et un unique point d'entrée Axios centralisant l'injection du JWT.

Dossier	Rôle
views/	Un composant par écran : LoginView, ProductListView, ProductFormView, ProductReviewView, NotFoundView.
components/	AppHeader, StepIndicator, RejectionBanner, et un composant par step du formulaire (ProductStep1 à 4).

Dossier	Rôle
stores/	État global Pinia : auth.store.js (token, rôle) et product.store.js (produit en cours d'édition).
services/	Instance Axios unique (api.js) : intercepteurs JWT + gestion des sessions expirées.
router/	Déclaration des routes + guard global de protection (authentification + rôle).

4.4 Modèle de données

Le modèle reste volontairement proche de l'existant fonctionnellement (mêmes champs métier), mais toutes les colonnes de contrôle (status, currentStep, createdBy, createdAt, updatedAt) sont désormais gérées exclusivement côté serveur, jamais saisies directement par le client.

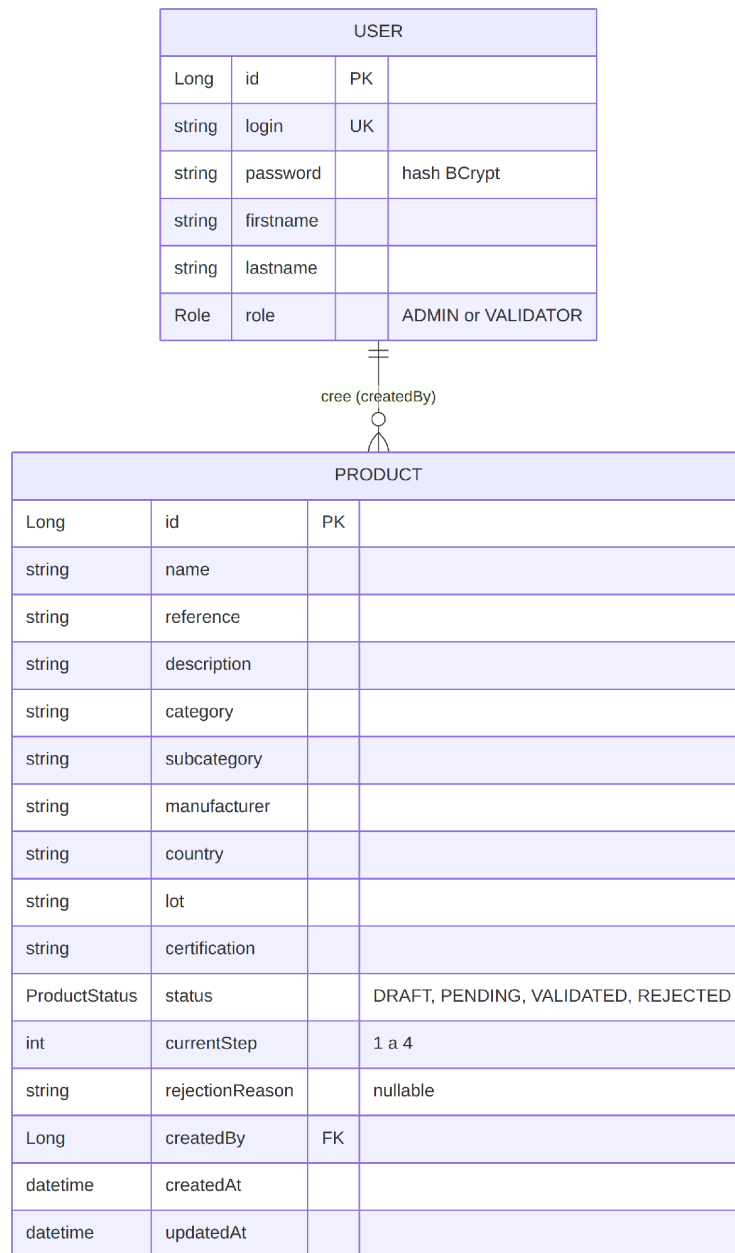


Figure 4 — Modèle de données : un User crée plusieurs Product ; le mot de passe est stocké sous forme de hash BCrypt.

5. Modèle fonctionnel et sécurité

5.1 Rôles et permissions

Le rôle USER de l'existant est supprimé : chaque compte doit désormais être explicitement ADMIN ou VALIDATOR, sans rôle intermédiaire ambigu.

Action	ADMIN	VALIDATOR
Se connecter	✓	✓
Voir la liste de tous les produits	✓	✓
Voir le détail d'un produit	✓	✓
Créer un nouveau produit	✓	✗
Éditer un produit (steps 1 à 4)	✓ (si DRAFT ou REJECTED)	✗ (lecture seule stricte)
Valider un produit	✗	✓ (si PENDING)
Refuser un produit avec motif	✗	✓ (si PENDING)

5.2 Cycle de vie d'un produit

Le fonctionnement en steps sans retour arrière est conservé (US-04.2), mais désormais entièrement revérifié côté serveur : le paramètre step n'est plus une valeur de confiance venant du client.

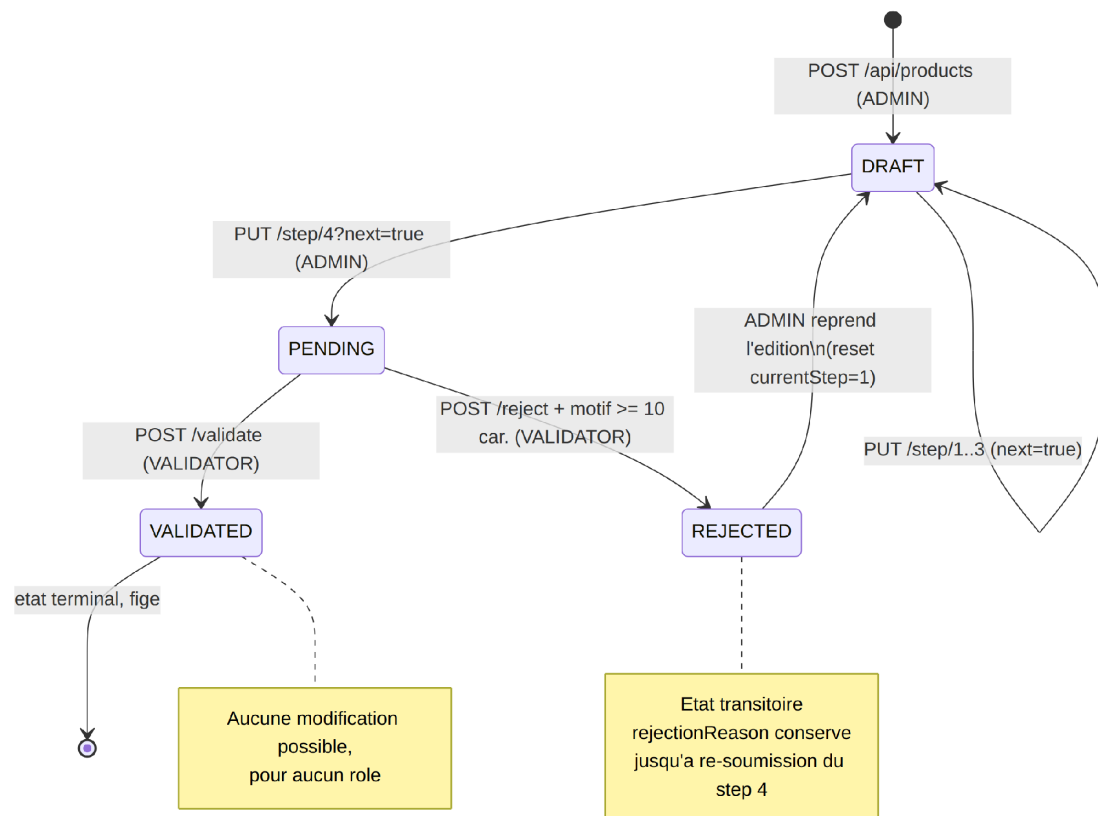


Figure 5 — Cycle de vie d'un produit : DRAFT → PENDING → VALIDATED (figé) ou REJECTED (transitoire, repasse en DRAFT à la reprise).

5.3 Authentification : séquence JWT

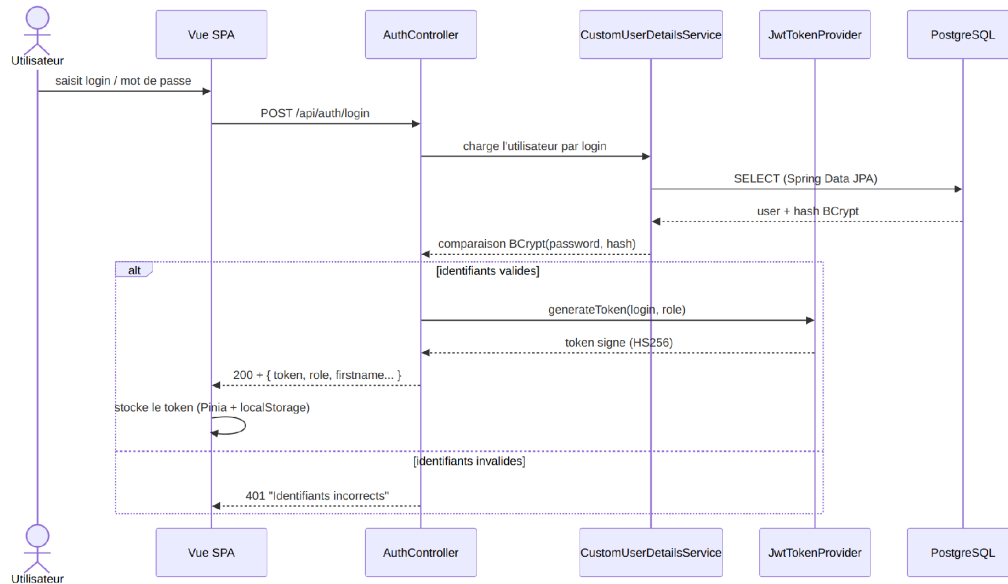


Figure 6 — Séquence de connexion : le mot de passe est comparé à son hash BCrypt, jamais stocké ni loggé en clair.

5.4 Workflow de création, soumission et décision

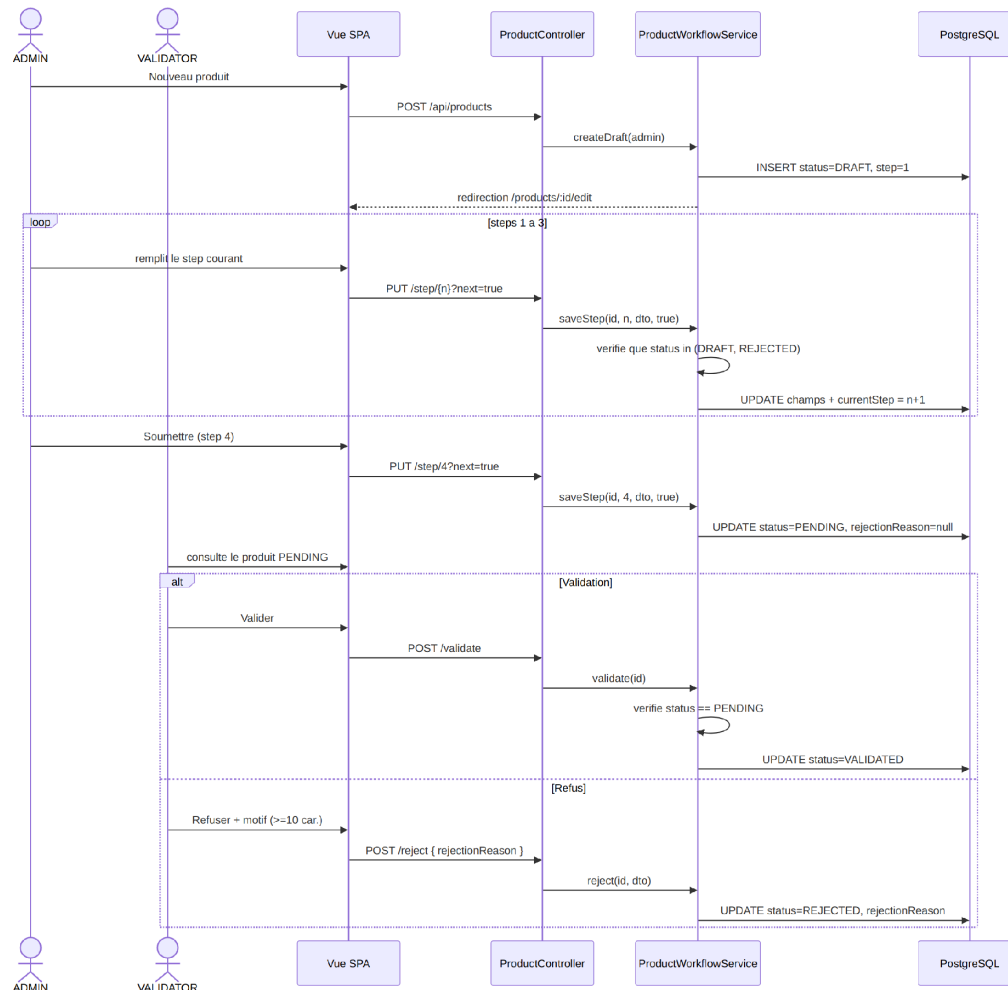


Figure 7 — Séquence complète : création du brouillon, progression step par step, soumission, puis décision du VALIDATOR (validation ou refus avec motif).

6. Correspondance entre bugs identifiés et solutions apportées

Cette table résume, pour chaque problème de l'existant, la mesure technique retenue dans la nouvelle architecture.

Problème existant	Solution dans DataCom v2
Injection SQL	Spring Data JPA (repositories), aucune requête SQL concaténée à la main.
Aucun contrôle de rôle serveur	@PreAuthorize("hasRole('ADMIN' 'VALIDATOR')") sur chaque endpoint sensible, rôle extrait du JWT signé, jamais fait confiance à une valeur client.
Mot de passe en clair en base	PasswordEncoder (BCrypt) fourni par Spring Security ; seul le hash est stocké.
Mot de passe loggé en clair	Plus de System.out.println(sql) ; les logs applicatifs ne manipulent plus de requêtes SQL brutes.
XSS stocké	Rendu Vue 3 (interpolation échappée par défaut) ; aucun contenu utilisateur injecté en HTML brut.
Fuite d'informations (debug info)	Aucune donnée de session exposée côté client ; les DTOs de réponse ne contiennent jamais le mot de passe.
Absence de CSRF	API stateless sans cookie de session ; l'authentification par en-tête Authorization n'est pas soumise au CSRF classique.
Credentials DB en dur	Connexion via variables d'environnement (DB_URL/DB_USER/DB_PASSWORD), jamais committées.
step non fiable côté serveur	ProductWorkflowService revalide systématiquement le statut et le step avant toute transition.
Stack trace exposée	GlobalExceptionHandler centralisé : chaque type d'erreur renvoie un message générique et un code HTTP adapté.
Rôle USER superflu	Enum Role limité à ADMIN et VALIDATOR ; plus de rôle intermédiaire ambigu.
Ressources DB non fermées	Spring Data JPA gère le cycle de vie des connexions (pool + transactions), plus de gestion manuelle de Connection/Statement.

7. Explication des briques de code les plus importantes

7.1 Sécurité — génération et validation du JWT

JwtTokenProvider centralise la création et la vérification des tokens. Le secret de signature n'est jamais codé en dur : il est injecté depuis une variable d'environnement (datacom.jwt.secret), directement inspiré du correctif appliqué au bug des credentials DB en dur.

```
@Component
public class JwtTokenProvider {

    @Value("${datacom.jwt.secret}")
    private String secret;

    @Value("${datacom.jwt.expiration-ms}")
    private long expirationMs;

    public String generateToken(String login, String role) {
        Date now = new Date();
        Date expiry = new Date(now.getTime() + expirationMs);
        return Jwts.builder()
```

```

        .subject(login)
        .claim("role", role)
        .issuedAt(now)
        .expiration(expiry)
        .signWith(getSigningKey())
        .compact();
    }
    // getRoleFromToken(), validateToken()...
}

```

JwtTokenProvider.java — le rôle est signé dans le token, jamais recalculé depuis une donnée envoyée par le client.

JwtAuthenticationFilter s'exécute avant chaque requête protégée : il lit l'en-tête Authorization, valide la signature, puis reconstruit l'autorité Spring Security (ROLE_ADMIN ou ROLE_VALIDATOR) à partir du contenu du token — jamais d'une valeur déclarée par le client.

```

String header = request.getHeader("Authorization");
if (header != null && header.startsWith("Bearer ")) {
    String token = header.substring(7);
    if (jwtTokenProvider.validateToken(token)) {
        String login = jwtTokenProvider.getLoginFromToken(token);
        String role = jwtTokenProvider.getRoleFromToken(token);
        var authorities = List.of(new SimpleGrantedAuthority("ROLE_" + role));
        var authentication = new UsernamePasswordAuthenticationToken(login, null, authorities);
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
}

```

JwtAuthenticationFilter.java — le rôle est revalidé à partir du token signé, à chaque requête.

SecurityConfig déclare la politique globale : API stateless (pas de session), tout endpoint sous /api/** requiert une authentification (sauf le login), et le filtre JWT est inséré avant le filtre d'authentification standard de Spring Security.

```

http.csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/login").permitAll()
        .requestMatchers("/api/**").authenticated()
        .anyRequest().denyAll()
    )
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

```

SecurityConfig.java — politique « deny by default » : tout ce qui n'est pas explicitement autorisé est refusé.

7.2 Machine à états du workflow produit

ProductWorkflowService est le cœur métier de l'application : il empêche toute transition de statut invalide, quelle que soit la donnée envoyée par le client. C'est cette classe qui règle directement le bug « step non fiable côté serveur » de l'existant.

```

public Product getEditableOrThrow(Long id) {
    Product product = getOrThrow(id);
    // N'importe quel ADMIN peut éditer n'importe quel produit,
    // tant qu'il est en DRAFT ou REJECTED.
    if (product.getStatus() != ProductStatus.DRAFT
        && product.getStatus() != ProductStatus.REJECTED) {
        throw new ForbiddenTransitionException(
            "Ce produit ne peut pas être modifié dans son état actuel");
    }
    return product;
}

public Product saveStep(Long id, int stepNumber, ProductStepRequest request, boolean moveNext) {
    Product product = getEditableOrThrow(id);

    // Reprise après refus : reset automatique en DRAFT, step 1
    if (product.getStatus() == ProductStatus.REJECTED) {
        product.setStatus(ProductStatus.DRAFT);
    }
}

```

```

        product.setCurrentStep(1);
    }
    applyStepFields(product, stepNumber, request);

    if (moveNext) {
        product.setCurrentStep(Math.min(stepNumber + 1, LAST_STEP));
        if (stepNumber == LAST_STEP) {
            product.setStatus(ProductStatus.PENDING);
            product.setRejectionReason(null);
        }
    }
    return productRepository.save(product);
}

```

ProductWorkflowService.java — la progression de step et les transitions de statut sont décidées uniquement côté serveur.

```

public Product validate(Long id) {
    Product product = getOrThrow(id);
    if (product.getStatus() != ProductStatus.PENDING) {
        throw new ForbiddenTransitionException(
            "Seul un produit en attente peut être validé");
    }
    product.setStatus(ProductStatus.VALIDATED);
    return productRepository.save(product);
}

public Product reject(Long id, ProductRejectRequest request) {
    Product product = getOrThrow(id);
    if (product.getStatus() != ProductStatus.PENDING) {
        throw new ForbiddenTransitionException(
            "Seul un produit en attente peut être refusé");
    }
    product.setStatus(ProductStatus.REJECTED);
    product.setRejectionReason(request.getRejectionReason());
    return productRepository.save(product);
}

```

ProductWorkflowService.java — validate() et reject() : la seule transition possible depuis PENDING est vers VALIDATED ou REJECTED.

7.3 Contrôle d'accès déclaratif au niveau du contrôleur

Chaque endpoint sensible est protégé par une annotation `@PreAuthorize`, lue et appliquée par Spring Security avant même l'exécution de la méthode. Le contrôle de rôle devient donc déclaratif et impossible à oublier silencieusement, contrairement à l'existant où `doGet/doPost` ne vérifiaient qu'une session active.

```

@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ProductDetailResponse create(Authentication authentication) { ... }

@PostMapping("/{id}/validate")
@PreAuthorize("hasRole('VALIDATOR')")
public ProductDetailResponse validate(@PathVariable Long id) { ... }

@PostMapping("/{id}/reject")
@PreAuthorize("hasRole('VALIDATOR')")
public ProductDetailResponse reject(@PathVariable Long id,
    @Valid @RequestBody ProductRejectRequest request) { ... }

```

ProductController.java — un ADMIN qui tente /validate reçoit un 403, avant même d'atteindre la logique métier.

7.4 Validation des DTOs d'entrée

`ProductRejectRequest` illustre comment une règle fonctionnelle précise (motif de refus obligatoire, 10 caractères minimum) est exprimée directement via Bean Validation, revérifiée côté serveur même si le formulaire Vue la vérifie déjà côté client.

```

@Data
public class ProductRejectRequest {
    @NotBlank
    @Size(min = 10, message = "Le motif de refus doit contenir au moins 10 caractères")

```

```
private String rejectionReason;
}
```

ProductRejectRequest.java — @Valid sur le contrôleur déclenche automatiquement un 400 si la contrainte n'est pas respectée.

7.5 Gestion centralisée des erreurs

GlobalExceptionHandler garantit qu'aucune exception non gérée ne remonte jusqu'au client sous forme de stack trace — le point faible le plus critique de l'existant après l'injection SQL.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ForbiddenTransitionException.class)
    public ResponseEntity<Object> handleForbiddenTransition(ForbiddenTransitionException ex) {
        return buildResponse(HttpStatus.CONFLICT, ex.getMessage());
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Object> handleGeneric(Exception ex) {
        log.error("Erreur inattendue", ex); // loggé côté serveur uniquement
        return buildResponse(HttpStatus.INTERNAL_SERVER_ERROR,
            "Une erreur inattendue est survenue"); // jamais la stack trace au client
    }
}
```

GlobalExceptionHandler.java — chaque type d'erreur métier a un code HTTP dédié ; le cas générique protège contre toute fuite d'information.

7.6 Entités : mot de passe hashé et rôles clos

```
@Entity @Table(name = "users")
public class User {
    @Column(nullable = false, unique = true)
    private String login;

    // Hash BCrypt, jamais le mot de passe en clair
    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role; // ADMIN ou VALIDATOR uniquement
}
```

User.java / Role.java — le rôle USER de l'existant n'existe simplement plus dans l'énumération : il ne peut plus être réintroduit par erreur.

7.7 Frontend — intercepteurs Axios

Toute la logique d'authentification côté frontend passe par un point d'entrée unique (services/api.js) : injection automatique du token sur chaque requête sortante, et gestion centralisée de l'expiration de session.

```
api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token')
    if (token) {
        config.headers = { ...config.headers, Authorization: `Bearer ${token}` }
    }
    return config
})

api.interceptors.response.use(
    (response) => response,
    async (error) => {
        if (error?.response?.status === 401) {
            const { useAuthStore } = await import('../stores/auth.store')
            useAuthStore().logout()
            router.push({ path: '/login', query: { message: 'Votre session a expiré...' } })
        }
        return Promise.reject(error)
    }
)
```

```
}
)
```

services/api.js — un seul endroit à maintenir pour toute la logique JWT côté client.

7.8 Frontend — garde de routes

Le router Vue applique un contrôle d'accès basé sur le rôle décodé du JWT, rappelé explicitement dans le code comme un confort UX et non une mesure de sécurité (la vraie protection reste côté API, via `@PreAuthorize`).

```
router.beforeEach((to, from, next) => {
  const token = localStorage.getItem('token')
  const authStore = useAuthStore()

  if (to.meta.requiresAuth && !token) {
    return next({ path: '/login' })
  }
  if (to.meta.role && authStore.role !== to.meta.role) {
    return next({ path: '/products' })
  }
  next()
})
```

router/index.js — un VALIDATOR qui force l'URL `/products/new` est redirigé, même si l'API refuserait de toute façon la requête POST correspondante.

8. Perspectives d'évolution

Plusieurs points ont été identifiés comme non bloquants pour la version actuelle mais à traiter dans une itération suivante :

- Pagination sur GET `/api/products`, pour anticiper la montée en volume.
- Adoption de SCSS ou Tailwind côté frontend pour homogénéiser les styles.
- Amélioration du schéma de base de données : contrainte d'unicité sur login sans duplicats, types varchar dimensionnés, colonne `is_active`.
- Mise en place de CI/CD (workflows, dependabot) pour sécuriser les dépendances dans la durée.
- Bascule de ddl-auto: update vers des migrations Flyway versionnées avant tout déploiement réel (décision volontairement différée, cf. backlog backend).
- Stratégie de révocation de JWT (refresh token ou blacklist serveur) si le besoin de déconnexion immédiate se confirme en usage réel.

Action jugée prioritaire à court terme : la dockerisation du frontend, pour aligner le mode de déploiement des deux applications.

9. Conclusion

La réécriture de DataCom illustre le passage d'un monolithe sans aucune séparation des responsabilités à une architecture en couches où chaque faille identifiée dans l'existant trouve une réponse structurelle et non un simple correctif ponctuel : Spring Security remplace l'absence de contrôle de rôle, Spring Data JPA remplace le SQL concaténé, BCrypt remplace le mot de passe en clair, et les DTOs remplacent l'exposition directe des entités.

Le fonctionnement métier validé par les utilisateurs — un workflow de création en 4 steps sans retour arrière, un cycle de vie produit en 4 statuts, deux rôles clairement délimités — a été conservé à l'identique, mais est désormais garanti côté serveur plutôt que simplement suggéré côté client.